
Libxmotion Implementation Notes

Ruixiang Du

ruixiang.du@gmail.com

July 22, 2026

ABSTRACT

These notes collect the core estimation and control formulations implemented in xmNavigation in one document, so that the shared conventions — coordinate frames, quaternion algebra, discrete-time propagation, and the scalar-templated numerical cores compiled by both the CPU and CUDA paths — carry a single consistent formulation across topics. The MEKF section derives the multiplicative extended Kalman filter for attitude and state estimation; the MPPI section derives the information-theoretic model-predictive path-integral controller and records its CPU and GPU execution strategy; the Platform Models section catalogs the benchmark plants these algorithms are evaluated on, each paired with the closed-form oracle that validates it. Standalone errata and implementation-review notes (e.g. the MEKF errata) are maintained separately.

Contents

1	MEKF	2
1.1	Quaternion	2
1.2	Error State Model	2
1.3	Measurement Model	5
1.4	MEKF Formulation	7
1.5	Initialization and Validation	9
2	MPPI	10
2.1	Lineage	10
2.2	Problem setup	10
2.3	Free energy and the optimal distribution	10
2.4	Importance sampling and the MPPI weights	11
2.5	The algorithm	11
2.6	Practical considerations	12
2.7	Implementation notes (CPU now, CUDA next)	12
3	Platform Models	13
3.1	Conventions	13
3.2	Wheeled platforms	13
3.3	Underactuated benchmark	14
3.4	Aerial platform	15
3.5	Legged platform	15
3.6	Choosing a model	15
	Bibliography	16

1 MEKF

This section is about the MEKF implementation in xmNavigation. The multiplicative extended Kalman filter originates with Lefferts, Markley and Shuster [1]; [2] discusses the attitude error representations it builds on, and [3] gives a widely used derivation of the quaternion indirect (error-state) Kalman filter. Most of the equations and derivation steps in this section are taken from [4], with the continuous-to-discrete noise derivation following Appendix E of [5].

1.1 Quaternion

The definition of a quaternion is given by:

$$\mathbf{q} = \begin{bmatrix} q_1 \\ \mathbf{q}_{2:4} \end{bmatrix} \quad (1)$$

The relevant operations on quaternions that are used for MEKF are given as follows:

- Multiplication:

$$\mathbf{q} \otimes \mathbf{p} = \begin{bmatrix} q_1 p_1 - \mathbf{q}_{2:4} \mathbf{p}_{2:4} \\ q_1 \mathbf{p}_{2:4} + p_1 \mathbf{q}_{2:4} + \mathbf{q}_{2:4} \times \mathbf{p}_{2:4} \end{bmatrix} \quad (2)$$

- Inverse:

$$\mathbf{q}^{-1} = \begin{bmatrix} q_1 \\ -\mathbf{q}_{2:4} \end{bmatrix} \quad (3)$$

A quaternion represents a rotation of one frame with respect to another. The inverse of a quaternion represents the rotation in the opposite direction. The multiplication of two quaternions represents the composition of two rotations.

We have the following properties of quaternions:

$$\mathbf{q} \otimes \mathbf{q}^{-1} = \begin{bmatrix} 1 \\ \mathbf{0} \end{bmatrix} \quad (4)$$

$$\begin{aligned} \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} \\ &= \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q} \end{aligned} \quad (5)$$

where $\boldsymbol{\omega}$ is the angular velocity vector of the body frame with respect to the inertial frame and $\boldsymbol{\Omega}$ is defined as

$$\boldsymbol{\Omega}(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\boldsymbol{\omega} \\ \boldsymbol{\omega} & -[\boldsymbol{\omega}_{\times}] \end{bmatrix} \quad (6)$$

$$\boldsymbol{\omega}_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \quad (7)$$

1.2 Error State Model

The MEKF is used to estimate the attitude of the body frame with respect to the inertial frame. The main difference between MEKF and other Kalman filters for attitude estimation is that MEKF does not use the quaternion in the state vector directly.

Normally, we would have the state dynamics as

$$\dot{\hat{\mathbf{q}}} = \frac{1}{2} \hat{\mathbf{q}} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} \quad (8)$$

$$\dot{\hat{\mathbf{v}}}^i = \mathbf{C}_b^i(\hat{\mathbf{q}}) \hat{\mathbf{f}}^b + \hat{\mathbf{g}}^i \quad (9)$$

$$\dot{\hat{\mathbf{r}}}^i = \hat{\mathbf{v}}^i \quad (10)$$

where $\mathbf{C}_b^i$ is the transformation matrix that transforms a vector from the body frame to the inertial frame, $\hat{\mathbf{q}}$ is the estimated quaternion, $\hat{\mathbf{v}}$ is the estimated velocity, and $\hat{\mathbf{r}}$ is the estimated position.

With MEKF, we use the error quaternion, which is the difference between the estimated quaternion and the true quaternion, as well as error velocity and position.

The error quaternion is given as

$$\mathbf{q} = \hat{\mathbf{q}} \otimes \delta \mathbf{q} \quad (11)$$

$$\Rightarrow \delta \mathbf{q} = \hat{\mathbf{q}}^{-1} \otimes \mathbf{q} \quad (12)$$

where $\hat{\mathbf{q}}$ is the estimated quaternion and $\delta \mathbf{q}$ is the error quaternion.

The error state vector is then defined as

$$\delta \mathbf{x} = \begin{bmatrix} \delta \mathbf{q} \\ \delta \mathbf{v} \\ \delta \mathbf{r} \\ \boldsymbol{\beta}_\omega \\ \boldsymbol{\beta}_f \\ \boldsymbol{\beta}_m \end{bmatrix} \quad (13)$$

where $\delta \mathbf{q}$ is the error quaternion, $\delta \mathbf{v}$ is the error in velocity, $\delta \mathbf{r}$ is the error in position, $\boldsymbol{\beta}_\omega$ is the bias in the angular velocity, $\boldsymbol{\beta}_f$ is the bias in the specific force, and $\boldsymbol{\beta}_m$ is the bias in the magnetometer.

Then we need to derive the dynamics of the error state $\dot{\delta \mathbf{x}}$. We can do it by examining each component of the state vector separately.

1.2.1 Error Quaternion Dynamics

The error quaternion dynamics is derived as follows:

$$\delta \mathbf{q} = \hat{\mathbf{q}}^{-1} \otimes \mathbf{q} \quad (14)$$

$$\Rightarrow \delta \dot{\mathbf{q}} = \hat{\mathbf{q}}^{-1} \otimes \dot{\mathbf{q}} + \dot{\hat{\mathbf{q}}}^{-1} \otimes \mathbf{q}$$

After the following steps described in [4], we will eventually get

$$\delta \dot{\mathbf{q}}_{2:4} \cong -\hat{\boldsymbol{\omega}}_\times \delta \mathbf{q}_{2:4} + \frac{1}{2} \delta \boldsymbol{\omega} \quad (15)$$

with the fact that $\delta q_1 = 1$ and the assumption that $\delta \mathbf{q}$ is small.

Here we replace the error states $\delta \mathbf{q}_{\{2:4\}}$ with a vector of small angles to further simplify the equations.

$$\begin{aligned} \boldsymbol{\alpha} &= 2\delta \mathbf{q}_{2:4} \\ \Rightarrow \dot{\boldsymbol{\alpha}} &= -\hat{\boldsymbol{\omega}}_\times \boldsymbol{\alpha} + \delta \boldsymbol{\omega} \end{aligned} \quad (16)$$

1.2.2 Error Velocity and Position Dynamics

For the velocity and position error, we have

$$\delta \mathbf{v} = \mathbf{v} - \hat{\mathbf{v}} \quad (17)$$

$$\delta \mathbf{r} = \mathbf{r} - \hat{\mathbf{r}} \quad (18)$$

Based on Equation 9 and Equation 10, we can derive the dynamics of the velocity and position error as

$$\delta \dot{\mathbf{v}} = -\mathbf{C}_b^i(\hat{\mathbf{q}}) \hat{\mathbf{f}}_\times^b \boldsymbol{\alpha} + \mathbf{C}_b^i \delta \mathbf{f} \quad (19)$$

$$\delta \dot{\mathbf{r}} = \delta \mathbf{v} \quad (20)$$

1.2.3 Sensor Bias Dynamics

The angular rate error model is given by

$$\boldsymbol{\omega} = \hat{\boldsymbol{\omega}} - \boldsymbol{\beta}_\omega - \boldsymbol{\eta}_\omega \quad (21)$$

$$\dot{\boldsymbol{\beta}}_\omega = \boldsymbol{\nu}_\omega \quad (22)$$

Together with the definition

$$\boldsymbol{\omega} = \hat{\boldsymbol{\omega}} + \delta \boldsymbol{\omega} \quad (23)$$

We can get

$$\delta \boldsymbol{\omega} = -\boldsymbol{\beta}_\omega - \boldsymbol{\eta}_\omega \quad (24)$$

$$\delta \dot{\boldsymbol{\omega}} = -\boldsymbol{\nu}_\omega \quad (25)$$

The linear acceleration error model is given by

$$\mathbf{f} = \hat{\mathbf{f}} - \boldsymbol{\beta}_f - \boldsymbol{\eta}_f \quad (26)$$

$$\dot{\boldsymbol{\beta}}_f = \boldsymbol{\nu}_f \quad (27)$$

Together with the definition

$$\mathbf{f} = \hat{\mathbf{f}} + \delta \mathbf{f} \quad (28)$$

We can get

$$\delta \mathbf{f} = -\boldsymbol{\beta}_f - \boldsymbol{\eta}_f \quad (29)$$

The magnetometer bias is treated as a slowly diverging random walk process driven by the noise process $\boldsymbol{\nu}_m$

$$\dot{\boldsymbol{\beta}}_m = \boldsymbol{\nu}_m \quad (30)$$

1.2.4 Full Error State Dynamics

According to the results above, we redefine the error state as

$$\delta \mathbf{x} = \begin{bmatrix} \boldsymbol{\alpha} \\ \delta \mathbf{v} \\ \delta \mathbf{r} \\ \boldsymbol{\beta}_\omega \\ \boldsymbol{\beta}_f \\ \boldsymbol{\beta}_m \end{bmatrix} \quad (31)$$

Combining equation Equation 16 Equation 19 Equation 20 Equation 25 Equation 27 Equation 30, together with Equation 24 and Equation 29, we get the full dynamics of the error state:

$$\delta \dot{\mathbf{x}} = \begin{bmatrix} \dot{\boldsymbol{\alpha}} \\ \delta \dot{\mathbf{v}} \\ \delta \dot{\mathbf{r}} \\ \dot{\boldsymbol{\beta}}_\omega \\ \dot{\boldsymbol{\beta}}_f \\ \dot{\boldsymbol{\beta}}_m \end{bmatrix} = \begin{bmatrix} -\hat{\boldsymbol{\omega}}_\times & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -\mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ -\mathbf{C}_b^i(\hat{\mathbf{q}}) \hat{\mathbf{f}}_\times^b & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -\mathbf{C}_b^i(\hat{\mathbf{q}}) & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha} \\ \delta \mathbf{v} \\ \delta \mathbf{r} \\ \boldsymbol{\beta}_\omega \\ \boldsymbol{\beta}_f \\ \boldsymbol{\beta}_m \end{bmatrix} + \begin{bmatrix} -\boldsymbol{\eta}_\omega \\ -\mathbf{C}_b^i(\hat{\mathbf{q}}) \boldsymbol{\eta}_f \\ \mathbf{0}_{3 \times 3} \\ \boldsymbol{\nu}_\omega \\ \boldsymbol{\nu}_f \\ \boldsymbol{\nu}_m \end{bmatrix} \quad (32)$$

The 18-error-state model is linear and time-varying with respect to the error states. Following the standard representation of a linear time-varying system, we can write the error state dynamics as

$$\delta\dot{\mathbf{x}} = \mathbf{F}\delta\mathbf{x} + \mathbf{G}\mathbf{w} \quad (33)$$

where matrices $\mathbf{F}$ is given by

$$\mathbf{F}(\hat{\mathbf{q}}, \hat{\boldsymbol{\omega}}, \hat{\mathbf{f}}) = \begin{bmatrix} -\hat{\boldsymbol{\omega}}_{\times} & \mathbf{0} & \mathbf{0} & -\mathbf{I}_{3 \times 3} & \mathbf{0} & \mathbf{0} \\ -\mathbf{C}_b^i(\hat{\mathbf{q}})\hat{\mathbf{f}}_{\times}^b & \mathbf{0} & \mathbf{0} & \mathbf{0} & -\mathbf{C}_b^i(\hat{\mathbf{q}}) & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{3 \times 3} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \quad (34)$$

the matrix $\mathbf{G}$ transforms the white noise sequence $\mathbf{w}$ into the disturbance vector and is given by

$$\mathbf{G} = \begin{bmatrix} -\mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\mathbf{C}_b^i(\hat{\mathbf{q}}) & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix} \quad (35)$$

The variance of the disturbance vector $\mathbf{w}$ is given by

$$\mathbf{Q}_c = \begin{bmatrix} \text{diag}(\boldsymbol{\sigma}_{\omega}^2) & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \text{diag}(\boldsymbol{\sigma}_f^2) & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \text{diag}(\boldsymbol{\sigma}_{\beta\omega}^2) & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \text{diag}(\boldsymbol{\sigma}_{\beta f}^2) & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \text{diag}(\boldsymbol{\sigma}_{\beta m}^2) \end{bmatrix} \quad (36)$$

where $\boldsymbol{\sigma}_{\omega}$, $\boldsymbol{\sigma}_f$, $\boldsymbol{\sigma}_{\beta\omega}$, $\boldsymbol{\sigma}_{\beta f}$, $\boldsymbol{\sigma}_{\beta m}$ are the standard deviations of the white noise processes. You can find more details about the derivation from Appendix E.1 of [5].

1.3 Measurement Model

The measurement model for the MEKF can be represented in the standard form

$$\delta\mathbf{z} = \mathbf{H}\delta\mathbf{x} + \mathbf{V} \quad (37)$$

where $\delta\mathbf{z}$ is the error measurement vector, $\mathbf{H}$ is the measurement function, $\delta\mathbf{x}$ is the error state vector, and $\mathbf{V}$ is the measurement noise.

In this case, the gyroscope measurement is treated as control input. We have the following two types of measurements:

- Accelerometer: $\tilde{\mathbf{a}}^b$
- Magnetometer: $\tilde{\mathbf{m}}^b$

In practice, $\tilde{\mathbf{a}}^b$ and $\tilde{\mathbf{m}}^b$ can be acquired directly from the accelerometer and magnetometer, respectively.

Note that Equation 37 in [4] will be used for the calculation of measurement residual for the accelerometer and magnetometer

$$\mathbf{C}_i^b(\mathbf{q}) \cong [\mathbf{I} - \boldsymbol{\alpha}_{\times}] \mathbf{C}_i^b(\hat{\mathbf{q}}) \quad (38)$$

1.3.1 Accelerometer Measurement Model

The accelerometer measurement model is given by:

$$\tilde{\mathbf{a}}^b = \mathbf{C}_i^b(\mathbf{q}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \boldsymbol{\beta}_f + \boldsymbol{\eta}_f \quad (39)$$

Here the acceleration of the rigid body where the IMU is attached to is neglected, and the accelerometer measurement is given by the gravity vector in the inertial frame rotated to the body frame.

Substituting Equation 38 into the above equation, we can get

$$\begin{aligned} \tilde{\mathbf{a}}^b &= [\mathbf{I} - \boldsymbol{\alpha}_\times] \mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \boldsymbol{\beta}_f + \boldsymbol{\eta}_f \\ &= \mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} - \boldsymbol{\alpha}_\times \mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \boldsymbol{\beta}_f + \boldsymbol{\eta}_f \\ &= \hat{\tilde{\mathbf{a}}}^b - \boldsymbol{\alpha} \times \mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \boldsymbol{\beta}_f + \boldsymbol{\eta}_f \\ &= \hat{\tilde{\mathbf{a}}}^b + \mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \times \boldsymbol{\alpha} + \boldsymbol{\beta}_f + \boldsymbol{\eta}_f \end{aligned} \quad (40)$$

$$\begin{aligned} \Rightarrow \delta \tilde{\mathbf{a}}^b &= \mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \times \boldsymbol{\alpha} + \boldsymbol{\beta}_f + \boldsymbol{\eta}_f \\ &= \left[\mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \times \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{I} \mathbf{0} \right] \begin{bmatrix} \boldsymbol{\alpha} \\ \delta \mathbf{v} \\ \delta \mathbf{r} \\ \boldsymbol{\beta}_\omega \\ \boldsymbol{\beta}_f \\ \boldsymbol{\beta}_m \end{bmatrix} + \boldsymbol{\eta}_f \end{aligned} \quad (41)$$

1.3.2 Magnetometer Measurement Model

The magnetometer measurement model is given by:

$$\tilde{\mathbf{m}}^b = \mathbf{C}_i^b(\mathbf{q}) \mathbf{m}^i + \boldsymbol{\beta}_m + \boldsymbol{\eta}_m \quad (42)$$

Similarly, substituting Equation 38, we can get

$$\begin{aligned} \tilde{\mathbf{m}}^b &\cong [\mathbf{I} - \boldsymbol{\alpha}_\times] \mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i + \boldsymbol{\beta}_m + \boldsymbol{\eta}_m \\ &= \mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i + \mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i \times \boldsymbol{\alpha} + \boldsymbol{\beta}_m + \boldsymbol{\eta}_m \\ &= \hat{\tilde{\mathbf{m}}}^b + \mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i \times \boldsymbol{\alpha} + \boldsymbol{\beta}_m + \boldsymbol{\eta}_m \end{aligned} \quad (43)$$

$$\begin{aligned}
\Rightarrow \delta \tilde{\mathbf{m}}^b &= \mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i \times \boldsymbol{\alpha} + \beta_m + \boldsymbol{\eta}_m \\
&= [\mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i \times \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{I}] \begin{bmatrix} \boldsymbol{\alpha} \\ \delta \mathbf{v} \\ \delta \mathbf{r} \\ \beta_\omega \\ \beta_f \\ \beta_m \end{bmatrix} + \boldsymbol{\eta}_m
\end{aligned} \tag{44}$$

1.3.3 Full Measurement Model

The full measurement model can be written as

$$\begin{bmatrix} \delta \tilde{\mathbf{a}}^b \\ \delta \tilde{\mathbf{m}}^b \end{bmatrix} = \begin{bmatrix} \left[\mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \times \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{I} \mathbf{0} \right] \\ \left[\mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i \times \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{I} \right] \end{bmatrix} \delta \mathbf{x} + \begin{bmatrix} \boldsymbol{\eta}_f \\ \boldsymbol{\eta}_m \end{bmatrix} \tag{45}$$

where $\mathbf{g}$ and $\mathbf{m}^i$ are the gravity vector and the magnetic field vector in the inertial frame, respectively and both are known constants.

We can define the measurement matrix $\mathbf{H}$ as

$$\mathbf{H} = \begin{bmatrix} \left[\mathbf{C}_i^b(\hat{\mathbf{q}}) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \times \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{I} \mathbf{0} \right] \\ \left[\mathbf{C}_i^b(\hat{\mathbf{q}}) \mathbf{m}^i \times \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{0} \mathbf{I} \right] \end{bmatrix} \tag{46}$$

The measurement noise covariance matrix $\mathbf{R}_c$ is given by

$$\mathbf{R} = \begin{bmatrix} \sigma_a^2 & \mathbf{0} \\ \mathbf{0} & \sigma_m^2 \end{bmatrix} \tag{47}$$

where σ_a and σ_m are the standard deviations of the accelerometer and magnetometer measurements, respectively.

1.4 MEKF Formulation

1.4.1 Model Discretization

We have acquired the error state dynamics and the measurement model as given in Equation 32 and Equation 45.

$$\delta \dot{\mathbf{x}} = \mathbf{F} \delta \mathbf{x} + \mathbf{G} \mathbf{w}$$

$$= \begin{bmatrix} -\hat{\boldsymbol{\omega}}_\times & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -\mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ -\mathbf{C}_b^i(\hat{\mathbf{q}}) \hat{\mathbf{f}}_\times^b & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -\mathbf{C}_b^i(\hat{\mathbf{q}}) & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha} \\ \delta \mathbf{v} \\ \delta \mathbf{r} \\ \beta_\omega \\ \beta_f \\ \beta_m \end{bmatrix} + \begin{bmatrix} -\boldsymbol{\eta}_\omega \\ -\mathbf{C}_b^i(\hat{\mathbf{q}}) \boldsymbol{\eta}_f \\ \mathbf{0}_{3 \times 3} \\ \boldsymbol{\nu}_\omega \\ \boldsymbol{\nu}_f \\ \boldsymbol{\nu}_m \end{bmatrix}$$

The state transition matrix for the discrete system, $\boldsymbol{\Phi}$, is given by:

$$\boldsymbol{\Phi} = e^{\mathbf{F} \Delta t} \tag{48}$$

The state transition matrix can be approximated as

$$\begin{aligned}
\Phi_{k-1} &= I + F_{k-1} \Delta t \\
&= I + \begin{bmatrix}
-\hat{\omega}_{k-1 \times} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -I_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
-C_b^i(\hat{q}_{k-1}) \hat{f}_{k-1 \times}^b & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -C_b^i(\hat{q}_{k-1}) & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & I_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3}
\end{bmatrix} \Delta t
\end{aligned} \tag{49}$$

Then the state covariance matrix can be updated as

$$P_k^- = \Phi_{k-1} P_{k-1}^+ \Phi_{k-1}^T + Q_d \tag{50}$$

where Q_d is the discrete-time process noise covariance matrix and is given by

$$Q_d = \int_0^{\Delta t} e^{F(t-\tau)} Q_c e^{F^T(t-\tau)} d\tau$$

$$= \begin{bmatrix}
\Lambda(\sigma_\omega^2) \Delta t + \Lambda(\sigma_{\beta\omega}^2) \frac{\Delta t^3}{3} & \mathbf{0} & \mathbf{0} & -\Lambda(\sigma_{\beta\omega}^2) \frac{\Delta t^2}{2} & \mathbf{0} & \mathbf{0} \\
\mathbf{0} & \Lambda(\sigma_f^2) \Delta t + \Lambda(\sigma_{\beta f}^2) \frac{\Delta t^3}{3} & \Lambda(\sigma_f^2) \Delta \frac{t^2}{2} + \Lambda(\sigma_{\beta f}^2) \frac{\Delta t^4}{8} & \mathbf{0} & -\Lambda(\sigma_{\beta f}^2) \Delta \frac{t^2}{2} & \mathbf{0} \\
\mathbf{0} & \Lambda(\sigma_f^2) \frac{\Delta t^2}{2} + \Lambda(\sigma_{\beta f}^2) \frac{\Delta t^4}{8} & \Lambda(\sigma_f^2) \frac{\Delta t^3}{3} + \Lambda(\sigma_{\beta f}^2) \frac{\Delta t^5}{20} & \mathbf{0} & -\Lambda(\sigma_{\beta f}^2) \frac{\Delta t^3}{6} & \mathbf{0} \\
-\Lambda(\sigma_{\beta\omega}^2) \frac{\Delta t^2}{2} & \mathbf{0} & \mathbf{0} & \Lambda(\sigma_{\beta\omega}^2) \Delta t & \mathbf{0} & \mathbf{0} \\
\mathbf{0} & -\Lambda(\sigma_{\beta f}^2) \Delta \frac{t^2}{2} & -\Lambda(\sigma_{\beta f}^2) \frac{\Delta t^3}{6} & \mathbf{0} & \Lambda(\sigma_{\beta f}^2) \Delta t & \mathbf{0} \\
\mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \Lambda(\sigma_{\beta m}^2) \Delta t
\end{bmatrix}$$

Similarly, we have the transition matrix for the measurement model as

$$H_k = \begin{bmatrix}
C_i^b(\hat{q}_k^-) \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} \times \mathbf{0} \ \mathbf{0} \ \mathbf{0} \ I \ \mathbf{0} \\
C_i^b(\hat{q}_k^-) m^i \times \mathbf{0} \ \mathbf{0} \ \mathbf{0} \ \mathbf{0} \ I
\end{bmatrix} \tag{51}$$

1.4.2 MEKF Algorithm

The MEKF can be formulated as follows:

- Initialize the filter with the initial state estimate $\delta \hat{x}_0^+$ and the initial error-state covariance matrix P_0^+

$$\delta \hat{x}_0^+ = \mathbf{0} \tag{52}$$

$$P_0^+ = E[(\delta x_0 - \delta \hat{x}_0^+)(\delta x_0 - \delta \hat{x}_0^+)^T] = E(\delta x_0 \delta x_0^T) \tag{53}$$

- For step $k = 1, 2, \dots$, perform the following steps:

- Update Φ_{k-1} using Equation 49

$$\Phi_{k-1} = I + F_{k-1} \Delta t$$

$$= I + \begin{bmatrix}
-\hat{\omega}_{k-1 \times} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -I_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
-C_b^i(\hat{q}_{k-1}) \hat{f}_{k-1 \times}^b & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & -C_b^i(\hat{q}_{k-1}) & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & I_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\
\mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3}
\end{bmatrix} \Delta t$$

- Predict the error state and error-state covariance matrix

$$P_k^- = \Phi_{k-1} P_{k-1}^+ \Phi_{k-1}^T + Q_d \tag{54}$$

$$\hat{x}_k^- = \Phi_{k-1} \hat{x}_{k-1} \tag{55}$$

- Update the measurement matrix $\mathbf{H}_k$ using Equation 51
- Compute the Kalman gain $\mathbf{K}_k$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (56)$$

- Update the error state and error-state covariance matrix

$$\delta \hat{\mathbf{x}}_k^+ = \delta \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\delta \mathbf{z}_k - \mathbf{H}_k \delta \hat{\mathbf{x}}_k^-) \quad (57)$$

Since the error state is reset to zero after every fold (see below), $\delta \hat{\mathbf{x}}_k^- = \mathbf{0}$ and the update reduces to $\delta \hat{\mathbf{x}}_k^+ = \mathbf{K}_k \delta \mathbf{z}_k$, with $\delta \mathbf{z}_k$ the measurement residual (innovation).

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- \quad (58)$$

- Update the full states

$$\hat{\mathbf{q}}_k^+ = \hat{\mathbf{q}}_k^- \otimes \begin{bmatrix} 1 \\ \frac{\boldsymbol{\alpha}_k^+}{2} \end{bmatrix} \quad (59)$$

$$\hat{\mathbf{r}}_k^+ = \hat{\mathbf{r}}_k^- + \delta \mathbf{r}_k^+ \quad (60)$$

$$\hat{\mathbf{v}}_k^+ = \hat{\mathbf{v}}_k^- + \delta \mathbf{v}_k^+ \quad (61)$$

$$\hat{\boldsymbol{\beta}}_{\omega,k}^+ = \hat{\boldsymbol{\beta}}_{\omega,k}^- + \boldsymbol{\beta}_{\omega,k}^+ \quad (62)$$

$$\hat{\boldsymbol{\beta}}_{f,k}^+ = \hat{\boldsymbol{\beta}}_{f,k}^- + \boldsymbol{\beta}_{f,k}^+ \quad (63)$$

$$\hat{\boldsymbol{\beta}}_{m,k}^+ = \hat{\boldsymbol{\beta}}_{m,k}^- + \boldsymbol{\beta}_{m,k}^+ \quad (64)$$

- Reset the error state for the next cycle: $\delta \hat{\mathbf{x}}_k^+ \leftarrow \mathbf{0}$. The bias terms must be folded into persistent nominal states here — the error-state estimate does not accumulate across cycles.

1.5 Initialization and Validation

Initialization. Starting the filter at identity with a large P_0 is both slow and, more importantly, outside the regime where a first-order filter is statistically honest (below). `attitude_init.hpp` provides the deterministic bootstrap: Shuster’s TRIAD [6] from a simultaneous accelerometer/magnetometer pair (full attitude, for MEKF9; the gravity pair anchors the primary axis), and accelerometer-only leveling (roll/pitch with zero yaw, the honest bootstrap for MEKF6, whose yaw is unobservable). `MagReferenceEnu` builds the inertial field reference from the local declination/inclination (e.g. from the World Magnetic Model for the deployment site); the ENU frame and sign conventions are documented in the header.

Statistical consistency (NEES). Both filters are validated by the standard normalized-estimation-error-squared criterion [7]: Monte-Carlo truth generated exactly per the filters’ noise model (continuous PSDs for gyro/accel noise, discrete observation covariances, random-walking biases), NEES computed on the $[\boldsymbol{\alpha}; \boldsymbol{\beta}_\omega]$ marginal against the filter’s own covariance blocks. Findings, recorded in `test_mekf_consistency.cpp`:

- Open-loop error-state propagation is exact: empirical attitude variance matches the propagated P to a ratio of 1.00.
- In the bootstrapped regime (attitude $\sigma \leq 3^\circ$, bias $\sigma \leq 0.01$), both filters are consistent, erring mildly conservative in the marginals; the joint NEES carries $\sim 1.4 \times$ optimism in the cross-correlations under sustained rotation — inherent first-order behavior, budgeted at $1.5 \times$ dof in the regression.

- With large initial uncertainty ($\sigma_\alpha = 0.1$ rad, $\sigma_{\beta\omega} = 0.1$ rad/s) the linearization breaks chi-square consistency ($\sim 2.4 \times$ overconfident attitude): initialize with TRIAD/leveling rather than relying on a large P_0 .
- Observability caveat: on a non-tumbling (planar) platform, yaw and the z gyro-bias are structurally unobservable to MEKF6 — their estimates wander and the covariance honestly grows. Use MEKF9 whenever heading matters; on tumbling trajectories all bias axes become observable and MEKF6 is consistent.

Remaining validation step (recorded, pending hardware data): replay of real IMU MCAP recordings as golden-log regressions.

2 MPPI

This section derives the MPPI control update implemented in `xmNavigation`, following the information-theoretic formulation, and records the implementation decisions (warm start, baseline subtraction, sampling variants, control parameterization) with references to the originating literature. It closes with the CPU execution strategy and the CUDA extension for discrete GPU and Jetson Orin platforms.

2.1 Lineage

Path-integral optimal control originates with Kappen [8], who showed that for control-affine stochastic systems with quadratic control cost the Hamilton–Jacobi–Bellman equation becomes linear under an exponential transformation of the value function, so the optimal control admits a Feynman–Kac path-integral representation evaluable by Monte Carlo. Theodorou et al. generalized the framework to parameterized policies as PI² [9]. Williams et al. brought it to receding-horizon control as MPPI, first with the path-integral derivation [10], [11], then re-derived it from an information-theoretic argument [12], [13] that removes the control-affine restriction — the dynamics only need to be **sampleable**. This note follows the information-theoretic derivation; the algorithm implemented in `src/control/mppi` is the algorithm of [13].

2.2 Problem setup

Consider discrete-time dynamics with state $\mathbf{x}_t \in \mathbb{R}^n$ and commanded control $\mathbf{v}_t \in \mathbb{R}^m$:

$$\mathbf{x}_{t+1} = \mathbf{F}(\mathbf{x}_t, \mathbf{v}_t) \quad (65)$$

where the **executed** control is the commanded control corrupted by (or deliberately injected with) Gaussian noise: $\mathbf{v}_t = \mathbf{u}_t + \boldsymbol{\varepsilon}_t$, $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$. A trajectory $\tau = (\mathbf{x}_0, \mathbf{v}_0, \dots, \mathbf{v}_{T-1})$ accrues the state cost

$$S(\tau) = \varphi(\mathbf{x}_T) + \sum_{t=0}^{T-1} q(\mathbf{x}_t) \quad (66)$$

with terminal cost φ and running cost q . Note S carries **no control cost**: the control effort penalty will emerge from the derivation as a KL term.

2.3 Free energy and the optimal distribution

Let $\mathbb{P}$ denote the **base** distribution over trajectories induced by running the noise alone ($\mathbf{u} = \mathbf{0}$), and $\mathbb{Q}_{\mathbf{u}}$ the distribution induced by a control sequence $\mathbf{u} = (\mathbf{u}_0, \dots, \mathbf{u}_{T-1})$. Define the **free energy** of the control problem [13]:

$$\mathcal{F}(S) = -\lambda \log \mathbb{E}_{\mathbb{P}} \left[\exp \left(-\frac{1}{\lambda} S(\tau) \right) \right] \quad (67)$$

where $\lambda > 0$ is the **inverse temperature**. Jensen’s inequality gives, for any distribution $\mathbb{Q}$ absolutely continuous with $\mathbb{P}$:

$$\mathcal{F}(S) \leq \mathbb{E}_{\mathbb{Q}}[S(\tau)] + \lambda \text{KL}(\mathbb{Q} \parallel \mathbb{P}) \quad (68)$$

i.e. the free energy lower-bounds the stochastic optimal control objective (state cost plus a control penalty in the form of a KL divergence from the base distribution). The bound is tight — Equation 68 holds with equality — for the **optimal distribution** $\mathbb{Q}^*$ defined by the Gibbs/Boltzmann density

$$\frac{d\mathbb{Q}^*}{d\mathbb{P}}(\tau) = \frac{\exp(-\frac{1}{\lambda} S(\tau))}{\mathbb{E}_{\mathbb{P}}[\exp(-\frac{1}{\lambda} S(\tau))]} \quad (69)$$

$\mathbb{Q}^*$ is not directly realizable by a controller. MPPI therefore picks the realizable distribution closest to it: minimize $\text{KL}(\mathbb{Q}^* \parallel \mathbb{Q}_{\mathbf{u}})$ over the mean-shifted Gaussian family. For Gaussian noise this minimization has the closed-form solution (moment matching)

$$\mathbf{u}_t^* = \mathbb{E}_{\mathbb{Q}^*}[\mathbf{v}_t] \quad (70)$$

2.4 Importance sampling and the MPPI weights

Equation 70 is an expectation under $\mathbb{Q}^*$, which we can only evaluate by importance sampling from a distribution we **can** roll out — in practice $\mathbb{Q}_{\hat{\mathbf{u}}}$, the distribution induced by the previous solution $\hat{\mathbf{u}}$ (warm start). Writing the expectation under $\mathbb{Q}_{\hat{\mathbf{u}}}$ and collecting the Radon–Nikodym factors of the Gaussian mean shift yields the **importance-sampling-corrected** trajectory cost [11], [13]:

$$\tilde{S}(\tau_k) = S(\tau_k) + \gamma \sum_{t=0}^{T-1} \hat{\mathbf{u}}_t^T \Sigma^{-1} \boldsymbol{\varepsilon}_{t,k} \quad (71)$$

where $\boldsymbol{\varepsilon}_{t,k}$ is the k -th sampled noise sequence and $\gamma = \lambda(1 - \alpha)$ exposes the **control-cost decoupling parameter** $\alpha \in [0, 1]$ introduced in [13] ($\alpha = 0$: full KL penalty toward the base distribution; $\alpha = 1$: no pull toward zero control). With K rollouts, the weights and update are

$$\rho = \min_k \tilde{S}(\tau_k), \quad w_k = \frac{\exp(-\frac{1}{\lambda} (\tilde{S}(\tau_k) - \rho))}{\sum_{j=1}^K \exp(-\frac{1}{\lambda} (\tilde{S}(\tau_j) - \rho))} \quad (72)$$

$$\mathbf{u}_t \leftarrow \hat{\mathbf{u}}_t + \sum_{k=1}^K w_k \boldsymbol{\varepsilon}_{t,k} \quad (73)$$

The baseline subtraction ρ leaves Equation 72 mathematically unchanged and is mandatory numerically (it prevents uniform exponent underflow) [13].

2.5 The algorithm

1. Shift the previous solution one step (executed step removed, tail padded); this is the sampling mean $\hat{\mathbf{u}}$.
2. Sample K noise sequences $\boldsymbol{\varepsilon}_k$; form perturbed controls $\mathbf{v}_k = \hat{\mathbf{u}} + \boldsymbol{\varepsilon}_k$ and clamp to actuator bounds.
3. Roll out the dynamics for each $\mathbf{v}_k$; accumulate Equation 66 and the correction of Equation 71.
4. Compute weights Equation 72 with baseline subtraction; apply the update Equation 73.
5. Optionally smooth the resulting sequence (Savitzky–Golay [14], or use a smoothness-preserving sampler, § below).

6. Execute $\mathbf{u}_0$; repeat.

The loop is a single optimization iteration per control step; the warm start is what makes this sufficient in practice [11].

2.6 Practical considerations

- **Temperature** λ sharpens ($\lambda \rightarrow 0$, winner-take-all) or flattens (large λ) the weights; it divides cost **differences**, so it is re-tuned whenever cost scales change [13].
- **Noise covariance** Σ is the exploration knob and defines the KL anchor; it must be full-rank on sampled channels since Σ^{-1} appears in Equation 71.
- **Failure modes and their literature**: sample impoverishment after disturbances (Tube-MPPI [15], Robust-MPPI [16]); control chattering from i.i.d. noise (Smooth-MPPI input lifting [17], low-frequency/colored sampling [18]); constraint handling — penalties inside rollouts are soft, so safety-critical constraints require an output-stage shield (control-barrier-function filtering [19]).
- **Sampling variants** implemented behind the sampler seam: normal-log-normal mixtures for cluttered spaces [20], colored noise [18], and annealed covariance schedules in the style of DIAL-MPC [21].
- **Control parameterization**: sampling spline knots instead of per-step controls reduces the decision dimension by an order of magnitude and enforces smoothness by construction; it is the enabling technique in whole-body legged sampling MPC [22], [23] and is a first-class option here.

2.7 Implementation notes (CPU now, CUDA next)

The CPU implementation follows the strategy validated by the Nav2 MPPI controller [14]: batch-major contiguous storage (structure-of-arrays over the K rollouts) that Eigen auto-vectorizes, preallocated workspace (no allocation in the control path), compile-time state/control dimensions via templates, and deterministic seeded sampling. Published throughput for this approach is roughly 30–60 million sample-timesteps per second per modern x86 core, which covers wheeled platforms (1–3k samples, 50–100 Hz) on a single core.

The rollout phase — clamp, step, accumulate cost — is extracted behind a backend seam (`rollout_backend.hpp`): sample k reads shared inputs and writes only S_k , so backends may evaluate the K rollouts in any parallel arrangement as long as the per-sample operation order is preserved (bitwise-reproducible results for any worker count; enforced by test). The CPU backend runs serially by default or over a pre-allocated worker pool with contiguous chunking. Measured on an 8-core x86 host at $K = 2048$: the SRB quadruped plan drops from 11.0 ms serial to 5.9 ms with 4 workers and 5.2 ms with 8; cheap wheeled models gain little (9.0 to 7.2 ms) because sequential noise generation dominates their budget (Amdahl). Per-sample counter-seeded noise streams would lift that cap and are the natural CPU follow-up; on the GPU path noise is generated on-device.

The CUDA backends implement the same seam following the architecture of MPPI-Generic [24]: one thread per sample (the split-rollout arrangement — right for models whose state fits in registers, 13 floats for the SRB), and dynamics/cost as scalar-templated raw-span cores (`model_core` / `critic_core`) that **both** backends compile — the Eigen model/critic classes wrap the cores on the CPU; a device-side “program” POD per platform family (wheeled, SRB quadruped; Section 3) calls them in float32. Device rollouts run in float because FP64

executes at 1/32 rate on GTX-class and Jetson Orin hardware; the seam stays double-typed (narrow on upload, widen on download), and CPU/GPU costs agree to float accumulation error (bounded by test at $10^{\{-3\}}$ relative, observed $\sim 10^{\{-5\}}$; the SRB program includes the quaternion integration and the world-inertia solve as $RD^{\{-1\}}R^T$). The SRB program ships the per-step foot plan and contact schedule with each call, so the per-cycle `SetContext` pattern is preserved.

A third backend moves sampling on-device: Gaussian noise from per-sample Philox streams [25] (counter-based, deterministic per seed and independent of thread scheduling) drawn inside the rollout kernel, with the weighted update $\sum_k w_k \varepsilon_k$ also computed on-device — `Plan()` then transfers only the costs down and the weights up (K floats each). Backends that sample on-device declare `kGeneratesNoise`; the controller skips its host sampler and fetches candidate noise for introspection on demand. End-to-end `Plan()` on a GTX 1660 Ti vs the 8-core host, wheeled model, $T = 50$: $K = 2048$: 0.14 ms device-sampling / 5.2 ms GPU-upload / 7.7 ms CPU-serial; $K = 131072$: 9.4 / 328 / 498 ms — host noise generation was the binding constraint, and with it removed six-figure sample counts fit a 100 Hz budget. SRB end-to-end ($T = 20$, host spline-knot sampling): $K = 2048$: 4.1 ms GPU vs 8.9 ms CPU-serial, now bounded by the host sampler — spline-knot generation on-device is the recorded follow-up, together with Jetson Orin deployment (unified memory: the pinned staging maps zero-copy).

3 Platform Models

This section documents the platform models implemented in `src/control/models` (namespace conventions, state/control layouts, governing equations, default parameters, and the closed-form oracle each implementation is validated against), with references to the originating literature. The set is chosen deliberately: classical models that are thoroughly understood in the literature yet rich enough to evaluate the estimation, planning, and control algorithms of this repository — each model stresses a property the others do not.

3.1 Conventions

Every model exposes the discrete `Step` concept used across the stack (`x_(t+1) = "Step"(x_t, u_t, t, Delta t)`, forward Euler unless noted) and, where the dynamics are continuous, the derivative `Deriv` used with the fixed-step classical RK4 propagator (`rk4.hpp`) for high-fidelity simulation. The numerical cores are scalar-templated raw-span functions in `model_core.hpp`, shared verbatim between the CPU implementations and the CUDA rollout programs (see Section 2). Units are SI (m, s, rad, N, kg) throughout; frames are stated per model. `linearize.hpp` provides central-difference Jacobians of any `Deriv` model about an operating point — the standard bridge to the linear tools (`SolveDlqr`, `StateFeedbackController`): linearize, Euler-discretize ($A_d = I + A\Delta t$, $B_d = B\Delta t$, consistent with `Step`), synthesize.

3.2 Wheeled platforms

3.2.1 Differential drive (kinematic unicycle)

State $\mathbf{x} = (x, y, \theta)$ (world position, heading), control $\mathbf{u} = (v, \omega)$ (forward speed, yaw rate):

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega \quad (74)$$

The model class of production wheeled MPPI deployments (Nav2); actuator dynamics are absorbed by the box constraints. File: `diff_drive.hpp`.

3.2.2 Kinematic bicycle (Ackermann)

State $\mathbf{x} = (x, y, \theta)$, control $\mathbf{u} = (v, \delta)$ (speed, front steering angle), wheelbase L :

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \left(\frac{v}{L}\right) \tan \delta \quad (75)$$

The standard low-speed car model [26]. Default $L = 0.5$ m (robot scale); the mismatch campaign overrides it to match the dynamic plant. File: `ackermann.hpp`.

3.2.3 Bicycle with acceleration input

State $\mathbf{x} = (x, y, v, \theta)$, control $\mathbf{u} = (a, \delta)$:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{v} = a, \quad \dot{\theta} = \left(\frac{v}{L}\right) \tan \delta \quad (76)$$

Successor of the retired `model/BicycleKinematics` (wheelbase constant $L = 2.4$ m preserved); the reachability Monte-Carlo propagates it with RK4. **Oracles:** straight-line constant-acceleration closed form; constant-steering circle of radius $L/\tan \delta$. File: `bicycle_accel.hpp`.

3.2.4 Single-track model with linear tires (“dynamic bicycle”)

The standard vehicle lateral-dynamics benchmark (Rajamani [27], ch. 2; the kinematic-vs-dynamic comparison plant of Kong et al. [26]). State $\mathbf{x} = (X, Y, \psi, v_x, v_y, r)$ — world position and heading, **body-frame** longitudinal/lateral velocities, yaw rate; control $\mathbf{u} = (a_x, \delta)$. With front/rear axle distances l_f, l_r from the CoM, mass m , yaw inertia I_z , and per-axle cornering stiffnesses C_f, C_r , the linear-tire slip angles and lateral forces are

$$\alpha_f = \frac{v_y + l_f r}{v_x} - \delta, \quad \alpha_r = \frac{v_y - l_r r}{v_x}, \quad F_{yf} = -C_f \alpha_f, \quad F_{yr} = -C_r \alpha_r \quad (77)$$

and the dynamics are

$$\dot{X} = v_x \cos \psi - v_y \sin \psi, \quad \dot{Y} = v_x \sin \psi + v_y \cos \psi, \quad \dot{\psi} = r \quad (78)$$

$$\dot{v}_x = a_x + v_y r, \quad \dot{v}_y = \frac{F_{yf} \cos \delta + F_{yr}}{m} - v_x r, \quad \dot{r} = \frac{l_f F_{yf} \cos \delta - l_r F_{yr}}{I_z} \quad (79)$$

The linear tire model is meaningless near standstill: slip-angle denominators clamp at $v_{x, \min}$ (default 0.5 m/s). Defaults are a mid-size passenger car: $m = 1500$ kg, $I_z = 2500 \text{ kg m}^2$, $l_f = 1.2$ m, $l_r = 1.6$ m, $C_f = C_r = 80000$ N/rad. **Oracle:** the steady-state yaw rate under constant speed and steering,

$$r_{ss} = \frac{v_x}{L + K_{us} v_x^2} \delta, \quad K_{us} = \frac{m}{L} \left(\frac{l_r}{C_f} - \frac{l_f}{C_r} \right), \quad L = l_f + l_r \quad (80)$$

verified to 1% by simulation with v_x held exactly ($a_x = -v_y r$); at low speed the model reproduces the kinematic yaw rate $(v/L) \tan \delta$ to 2%. Role in this repository: the designated **mismatch plant** — planners use the kinematic bicycle, this plant slips (per-seed tire-stiffness variation in the nightly campaign). File: `dynamic_bicycle.hpp`.

3.3 Underactuated benchmark

3.3.1 Cart-pole (inverted pendulum on a cart)

The classical underactuated benchmark, in the exact formulation of Barto, Sutton & Anderson [28] (uniform pole, frictionless track and pivot). State $\mathbf{x} = (x, \dot{x}, \theta, \dot{\theta})$ with θ measured **from upright** ($\theta = 0$ is the unstable equilibrium, $\theta = \pi$ hangs down); control $u = F$ (force on the cart). With cart mass M , pole mass m , pivot-to-CoM distance l (half the pole length):

$$\ddot{\theta} = \frac{g \sin \theta - \cos \theta \cdot \xi}{l \left(\frac{4}{3} - \frac{m \cos^2 \theta}{M+m} \right)}, \quad \xi = \frac{F + m l \dot{\theta}^2 \sin \theta}{M + m} \quad (81)$$

$$\ddot{x} = \xi - \frac{m l \ddot{\theta} \cos \theta}{M + m} \quad (82)$$

The 4/3 factor carries the uniform pole's moment of inertia ($I_{\text{com}} = m(2l)^2/12$). Defaults are the literature-standard set: $M = 1$ kg, $m = 0.1$ kg, $l = 0.5$ m, force limit ± 10 N. **Oracles:** conservation of the mechanical energy

$$E = \frac{1}{2} M \dot{x}^2 + \frac{1}{2} m (\dot{x} + l \dot{\theta} \cos \theta)^2 + \frac{1}{2} m (l \dot{\theta} \sin \theta)^2 + \frac{1}{2} I_{\text{com}} \dot{\theta}^2 + m g l (\cos \theta - 1) \quad (83)$$

under zero force ($|\Delta E| < 10^{-6}$ relative over 10 s of RK4), instability of the upright equilibrium, and bounded oscillation about the hanging one. The integration scenario is the classical controller handover: MPPI swings the pole up globally, a DLQR state-feedback controller (gain synthesized from a numeric linearization at the upright) catches and holds. File: `cartpole.hpp`.

3.4 Aerial platform

3.4.1 Quadrotor (rigid body)

The canonical aerial benchmark [29]. State (13): $\mathbf{p} \in \mathbb{R}^3$, $\mathbf{v} \in \mathbb{R}^3$ (world frame), unit quaternion $\mathbf{q}$ (w-x-y-z, body-to-world), $\boldsymbol{\omega} \in \mathbb{R}^3$ in the **body** frame — note the SRB quadruped model (Section 2) uses world-frame $\boldsymbol{\omega}$; the difference is deliberate and follows each model's literature convention. Control $\mathbf{u} = (f, \boldsymbol{\tau})$: total thrust along the body z axis and body-frame moments. With diagonal inertia $\mathbf{I}$:

$$\dot{\mathbf{p}} = \mathbf{v}, \quad \dot{\mathbf{v}} = -g \mathbf{e}_3 + \left(\frac{f}{m} \right) \mathbf{R}(\mathbf{q}) \mathbf{e}_3 \quad (84)$$

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes (0, \boldsymbol{\omega}), \quad \dot{\boldsymbol{\omega}} = \mathbf{I}^{-1} (\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{I} \boldsymbol{\omega}) \quad (85)$$

Step renormalizes the quaternion after the Euler update. Defaults are a lab-scale small quadrotor: $m = 0.5$ kg, $\mathbf{I} = \text{diag}(2.32, 2.32, 4.0) \times 10^{-3} \text{kg m}^2$. **Oracles:** hover ($f = mg$, level) is an exact equilibrium; free fall matches $z = -gt^2/2$; a pure yaw moment integrates $\omega_z = (\tau_z/I_{zz})t$ exactly (the gyroscopic term vanishes for pure z rotation) with the quaternion norm preserved. File: `quadrotor.hpp`.

3.5 Legged platform

The single-rigid-body (SRB) quadruped — trunk as one rigid body driven by per-foot ground-reaction forces, massless legs, per-step foot plan and contact schedule — is documented in Section 2 (it is that controller's primary legged plant) and follows the convex-MPC lineage [30]. File: `srb_quadruped.hpp`.

3.6 Choosing a model

Model	Stresses	Typical use
double integrator	analytic oracles, LQR baselines	controller unit tests
diff drive / kinematic bicycle	planar planning, box constraints	MPPI wheeled scenarios, tuner

bicycle (accel input)	second-order wheeled, RK4 propagation	reachability Monte-Carlo
dynamic bicycle	model mismatch , slip, speed-dependent stability	nightly robustness campaigns
cart-pole	instability, swing-up/catch handover	global+local controller composition
quadrotor	3D attitude, force/moment control	3D estimation + control in the loop
SRB quadruped	contact schedules, GRF control	legged MPPI, GPU rollouts

Bibliography

- [1] E. J. Lefferts, F. L. Markley, and M. D. Shuster, “Kalman Filtering for Spacecraft Attitude Estimation,” *Journal of Guidance, Control, and Dynamics*, vol. 5, no. 5, pp. 417–429, 1982, doi: [10.2514/3.56190](https://doi.org/10.2514/3.56190).
- [2] F. L. Markley, “Attitude Error Representations for Kalman Filtering,” *Journal of Guidance, Control, and Dynamics*, vol. 26, no. 2, pp. 311–317, 2003, doi: [10.2514/2.5048](https://doi.org/10.2514/2.5048).
- [3] N. Trawny and S. I. Roumeliotis, “Indirect Kalman Filter for 3D Attitude Estimation,” technical report 2005–2, 2005. [Online]. Available: <http://mars.cs.umn.edu/tr/reports/Trawny05b.pdf>
- [4] J. M. Maley, “Multiplicative quaternion extended kalman filtering for nonspinning guided projectiles,” technical report, 2013. [Online]. Available: <https://apps.dtic.mil/sti/pdfs/ADA588831.pdf>
- [5] J. Solà, “Quaternion kinematics for the error-state Kalman filter,” Nov. 2017, [Online]. Available: <http://arxiv.org/abs/1711.02508>
- [6] M. D. Shuster and S. D. Oh, “Three-Axis Attitude Determination from Vector Observations,” *Journal of Guidance and Control*, vol. 4, no. 1, pp. 70–77, 1981, doi: [10.2514/3.19717](https://doi.org/10.2514/3.19717).
- [7] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. Wiley, 2001. doi: [10.1002/0471221279](https://doi.org/10.1002/0471221279).
- [8] H. J. Kappen, “Linear Theory for Control of Nonlinear Stochastic Systems,” *Physical Review Letters*, vol. 95, p. 200201, 2005, doi: [10.1103/PhysRevLett.95.200201](https://doi.org/10.1103/PhysRevLett.95.200201).
- [9] E. Theodorou, J. Buchli, and S. Schaal, “A Generalized Path Integral Control Approach to Reinforcement Learning,” *Journal of Machine Learning Research*, vol. 11, pp. 3137–3181, 2010.
- [10] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive Driving with Model Predictive Path Integral Control,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1433–1440. doi: [10.1109/ICRA.2016.7487277](https://doi.org/10.1109/ICRA.2016.7487277).

- [11] G. Williams, A. Aldrich, and E. A. Theodorou, “Model Predictive Path Integral Control: From Theory to Parallel Computation,” *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017, doi: [10.2514/1.G001921](https://doi.org/10.2514/1.G001921).
- [12] G. Williams *et al.*, “Information Theoretic MPC for Model-Based Reinforcement Learning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 1714–1721. doi: [10.1109/ICRA.2017.7989202](https://doi.org/10.1109/ICRA.2017.7989202).
- [13] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving,” *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018, doi: [10.1109/TRO.2018.2865891](https://doi.org/10.1109/TRO.2018.2865891).
- [14] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, “From the Desks of ROS Maintainers: A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2.” 2023.
- [15] G. Williams, B. Goldfain, P. Drews, K. Saigol, J. M. Rehg, and E. A. Theodorou, “Robust Sampling Based Model Predictive Control with Sparse Objective Information,” in *Robotics: Science and Systems XIV*, 2018. doi: [10.15607/RSS.2018.XIV.042](https://doi.org/10.15607/RSS.2018.XIV.042).
- [16] M. Gandhi, B. Vlahov, J. Gibson, G. Williams, and E. A. Theodorou, “Robust Model Predictive Path Integral Control: Analysis and Performance Guarantees,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1423–1430, 2021.
- [17] T. Kim, G. Park, K. Kwak, J. Bae, and W. Lee, “Smooth Model Predictive Path Integral Control Without Smoothing,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10406–10413, 2022.
- [18] B. Vlahov, J. Gibson, D. D. Fan, P. Spieler, A.-a. Agha-mohammadi, and E. A. Theodorou, “Low Frequency Sampling in Model Predictive Path Integral Control,” *IEEE Robotics and Automation Letters*, vol. 9, no. 5, 2024, doi: [10.1109/LRA.2024.3382530](https://doi.org/10.1109/LRA.2024.3382530).
- [19] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, “Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Method Using Control Barrier Functions,” *IEEE Robotics and Automation Letters*, vol. 8, no. 11, 2023.
- [20] I. S. Mohamed, K. Yin, and L. Liu, “Autonomous Navigation of AGVs in Unknown Cluttered Environments: log-MPPI Control Strategy,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10240–10247, 2022.
- [21] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, “Full-Order Sampling-Based MPC for Torque-Level Locomotion Control via Diffusion-Style Annealing.” 2024.
- [22] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, “Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo.” 2022.
- [23] J. Alvarez-Padilla, J. Z. Zhang, S. Kwok, J. M. Dolan, and Z. Manchester, “Real-Time Whole-Body Control of Legged Robots with Model-Predictive Path Integral Control.” 2024.
- [24] B. Vlahov, J. Gibson, M. Gandhi, and E. A. Theodorou, “MPPI-Generic: A CUDA Library for Stochastic Trajectory Optimization.” 2024.

- [25] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, “Parallel Random Numbers: As Easy as 1, 2, 3,” in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, 2011, pp. 16:1–16:12. doi: [10.1145/2063384.2063405](https://doi.org/10.1145/2063384.2063405).
- [26] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, “Kinematic and Dynamic Vehicle Models for Autonomous Driving Control Design,” in *IEEE Intelligent Vehicles Symposium (IV)*, 2015, pp. 1094–1099. doi: [10.1109/IVS.2015.7225830](https://doi.org/10.1109/IVS.2015.7225830).
- [27] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. Springer, 2011. doi: [10.1007/978-1-4614-1433-9](https://doi.org/10.1007/978-1-4614-1433-9).
- [28] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, no. 5, pp. 834–846, 1983, doi: [10.1109/TSMC.1983.6313077](https://doi.org/10.1109/TSMC.1983.6313077).
- [29] D. Mellinger and V. Kumar, “Minimum Snap Trajectory Generation and Control for Quadrotors,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 2520–2525. doi: [10.1109/ICRA.2011.5980409](https://doi.org/10.1109/ICRA.2011.5980409).
- [30] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–9. doi: [10.1109/IROS.2018.8594448](https://doi.org/10.1109/IROS.2018.8594448).